

SOLID Principles — Simple English + Simple Examples

SOLID is five rules for writing classes that are easy to change without breaking things. The goal of all five: **when requirements change, you edit in one place, not ten.**

Letter	Name	One-line meaning
S	Single Responsibility	A class should do one job.
O	Open/Closed	Add new behaviour without editing old code.
L	Liskov Substitution	A subclass must work anywhere its parent works.
I	Interface Segregation	Don't force a class to implement methods it doesn't need.
D	Dependency Inversion	Depend on interfaces, not concrete classes.

S — Single Responsibility Principle

A class should have only one reason to change. One job. If a class does two unrelated things, a change to one can break the other.

Bad — this class does three jobs

```
kotlin
class User {
    fun saveToDatabase() { /* database code */ }
    fun sendEmail() { /* email code */ }
    fun validateInput() { /* validation code */ }
}
```

If the database changes, you edit `User`. If email changes, you edit `User`. If validation changes, you edit `User`. Three reasons to change one class.

Good — split by job

```
kotlin
```

```

class User(val name: String, val email: String)

class UserRepository {
    fun save(user: User) { /* database code */ }
}

class EmailService {
    fun sendWelcome(user: User) { /* email code */ }
}

class UserValidator {
    fun validate(user: User): Boolean { /* validation code */ }
}

```

Now each class has one reason to change. Database change → only `UserRepository`.

Android example: This is exactly why we split into layers. The ViewModel handles UI state, the Repository handles data, the Use Case handles business rules. Each has one job. A `MatchCard` composable draws a match — it doesn't fetch data.

Memory hook: *"One class, one job."*

O — Open/Closed Principle

Open for extension, closed for modification. You should be able to add new behaviour by adding new code, not by editing existing, working code. Editing tested code risks breaking it.

Bad — every new shape means editing this function

```

kotlin

class AreaCalculator {
    fun area(shape: Any): Double {
        return when (shape) {
            is Circle -> 3.14 * shape.radius * shape.radius
            is Square -> shape.side * shape.side
            // add Triangle? You must EDIT this function again.
            else -> 0.0
        }
    }
}

```

Every new shape forces you to open and change `AreaCalculator`.

Good — new shapes just implement the interface

```

kotlin

```

```

interface Shape {
    fun area(): Double
}

class Circle(val radius: Double) : Shape {
    override fun area() = 3.14 * radius * radius
}

class Square(val side: Double) : Shape {
    override fun area() = side * side
}

// New shape? Add a NEW class. Don't touch existing code.
class Triangle(val base: Double, val height: Double) : Shape {
    override fun area() = 0.5 * base * height
}

class AreaCalculator {
    fun area(shape: Shape) = shape.area()    // never changes
}

```

`AreaCalculator` never changes again. You *extend* by adding classes.

Android example: A `RecyclerView.Adapter` with different view types, or a sealed class of UI states — you add a new state/type without rewriting the handler. Strategy pattern is the classic O/C tool.

Memory hook: *"Add new code, don't change old code."*

L — Liskov Substitution Principle

A subclass must be usable anywhere its parent is used, without surprises. If code works with the parent type, swapping in a child must not break it.

Bad — the child breaks the parent's promise

kotlin

```

open class Bird {
    open fun fly() { println("Flying") }
}

class Sparrow : Bird() // fine, sparrows fly

class Ostrich : Bird() {
    override fun fly() {
        throw Exception("Ostriches can't fly!") // breaks the contract
    }
}

```

Any code that does `bird.fly()` will crash if handed an `Ostrich`. The `Ostrich` is *not* a valid substitute for `Bird`, so this hierarchy is wrong.

Good — model the hierarchy honestly

```

kotlin

open class Bird {
    open fun eat() { println("Eating") }
}

interface CanFly {
    fun fly()
}

class Sparrow : Bird(), CanFly {
    override fun fly() { println("Flying") }
}

class Ostrich : Bird() {
    // no fly() - an ostrich simply isn't a flying bird
}

```

Now anything typed `CanFly` truly can fly. No subclass breaks a promise.

The test: if you have to write "throws exception" or "does nothing" in an override to escape the parent's behaviour, the hierarchy is probably wrong.

Android example: If a base `Repository` promises `getData(): List<T>`, a subclass shouldn't override it to return null or throw. Callers rely on the parent's contract.

Memory hook: *"A child must keep the parent's promises."*

I — Interface Segregation Principle

Don't force a class to implement methods it doesn't use. Many small, focused interfaces beat one big fat interface.

Bad — one big interface forces unused methods

kotlin

```
interface Worker {
    fun work()
    fun eat()
    fun sleep()
}

class Human : Worker {
    override fun work() { }
    override fun eat() { }
    override fun sleep() { }
}

class Robot : Worker {
    override fun work() { }
    override fun eat() { /* robots don't eat – forced to implement anyway */ }
    override fun sleep() { /* robots don't sleep – empty, awkward */ }
}
```

`Robot` is forced to implement `eat()` and `sleep()` it doesn't need.

Good — split into focused interfaces

kotlin

```
interface Workable { fun work() }
interface Eatable { fun eat() }
interface Sleepable { fun sleep() }

class Human : Workable, Eatable, Sleepable {
    override fun work() { }
    override fun eat() { }
    override fun sleep() { }
}

class Robot : Workable {
    override fun work() { } // only what it actually does
}
```

`Robot` implements only `Workable`. No empty, meaningless methods.

Android example: Instead of one giant `ClickListener` with ten callbacks, give each interaction its own small interface (or lambda). A composable that only needs an `onClick` shouldn't be handed an interface with `onLongClick`, `onSwipe`, `onDrag` too.

Memory hook: *"Many small interfaces, not one fat one."*

D — Dependency Inversion Principle

Depend on abstractions (interfaces), not on concrete classes. High-level code shouldn't be nailed to a specific low-level implementation.

Bad — ViewModel is welded to a concrete class

kotlin

```
class MatchesViewModel {  
    // directly creates a concrete class - can't swap it, can't test it  
    private val repository = MatchRepositoryImpl()  
  
    fun load() = repository.getMatches()  
}
```

`MatchesViewModel` is stuck with `MatchRepositoryImpl`. You can't swap in a fake for testing, or a different implementation later, without editing the ViewModel.

Good — depend on the interface

kotlin

```
interface MatchRepository {  
    fun getMatches(): List<Match>  
}  
  
class MatchRepositoryImpl : MatchRepository {  
    override fun getMatches(): List<Match> { /* real data */ return emptyList()  
}  
  
class FakeMatchRepository : MatchRepository {  
    override fun getMatches(): List<Match> = listOf(/* test data */)  
}  
  
class MatchesViewModel(  
    private val repository: MatchRepository // the interface, injected  
) {  
    fun load() = repository.getMatches()  
}
```

Now the ViewModel depends on the `MatchRepository` interface. You can pass the real one in production, a fake one in tests. Swap implementations freely.

This is the whole basis of Clean Architecture: the domain declares the `MatchRepository` interface, the data layer implements it, and the ViewModel depends only on the interface. That's dependency inversion. Hilt (or a manual factory) supplies the concrete class from the outside.

Memory hook: *"Depend on the interface, not the concrete class."*

Putting it together — how they connect

- **S** keeps each class small and focused.
- **O** lets you add features by adding classes, not editing old ones.
- **L** makes inheritance safe — subclasses behave.
- **I** keeps interfaces small so nobody implements junk they don't need.
- **D** points dependencies at interfaces, so you can swap and test.

S, O, and D show up constantly in Android:

- Layered architecture (ViewModel / UseCase / Repository) = **S**
 - Sealed UI states, strategy patterns = **O**
 - Domain interface + data implementation + injection = **D**
-

30-second interview version of each

- **S — Single Responsibility:** *"One class, one job. Split data, UI, and validation into separate classes so each has one reason to change."*
- **O — Open/Closed:** *"Add new behaviour with new classes, don't edit working code. Use interfaces so a new type just implements it."*
- **L — Liskov Substitution:** *"A subclass must work anywhere the parent works. If an override has to throw or do nothing to escape the parent's behaviour, the hierarchy is wrong."*
- **I — Interface Segregation:** *"Don't force a class to implement methods it doesn't use. Prefer many small interfaces over one fat one."*
- **D — Dependency Inversion:** *"Depend on interfaces, not concrete classes. The ViewModel depends on a Repository interface, and the concrete implementation is injected — that's the basis of Clean Architecture."*